Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Computer Vision and Image Processing (CSCS111/CSDS119)
Lab Assignment 9
Depth Analysis

1 Introduction and Problem Formulation

In stereo vision, 3D depth information of a real-world scene is recovered from a pair of 2D images captured from two horizontally displaced viewpoints. When the stereo image pair is rectified, epipolar lines become parallel to the horizontal image scanlines. Consequently, the correspondence search space is simplified from a 2D plane to a 1D horizontal row.

1.1 Camera Geometry and Depth Triangulation

For a calibrated and rectified pinhole stereo camera system, depth Z is calculated using the standard triangulation formula:

$$Z = \frac{f \cdot B}{d} \tag{1}$$

where:

- Z is the perpendicular distance (depth) from the camera baseline to the scene point in metres (m).
- f is the camera focal length in pixels ($f = 800$ px).
- B is the baseline separation between optical camera centres ($B = 0.20$ m).
- $d = x_L - x_R$ is the horizontal disparity in pixels.

Given $f = 800$ px and $B = 0.20$ m, the camera constant is:

$$f \cdot B = 800 \times 0.20 = 160.0 \quad (\text{px} \cdot \text{m}) \tag{2}$$

The scene contains three synthetic objects with designed disparities:

- **Near Object (Blue Box)**: Designed disparity $d \approx 80$ px $\implies Z_{\text{theo}} = \frac{160}{80} = 2.00$ m.
- **Middle Object (Green Box)**: Designed disparity $d \approx 40$ px $\implies Z_{\text{theo}} = \frac{160}{40} = 4.00$ m.
- **Far Object (Orange Box)**: Designed disparity $d \approx 20$ px $\implies Z_{\text{theo}} = \frac{160}{20} = 8.00$ m.

1.2 Environment Setup

The implementation is executed in Python using OpenCV (`cv2`), NumPy, and Matplotlib.

Listing 1: Setup and Camera Constants Initialization

```
import os
import warnings
os.environ['MPLCONFIGDIR'] = '/tmp/matplotlib'
warnings.filterwarnings('ignore')

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Camera Parameters given in the problem
f = 800.0   # Focal length in pixels
B = 0.20    # Baseline in metres
fB = f * B  # 160.0 (pixels * metres)

print(f"Focal length f: {f} px")
print(f"Baseline B: {B} m")
print(f"f * B constant: {fB} px*m")
```

Listing 2: Console Output of Camera Constants

```
Focal length f: 800.0 px
Baseline B: 0.2 m
f * B constant: 160.0 px*m
```

2 Step 1: Load and Display Stereo Pair in Grayscale

The input rectified stereo pair (`stereo_left.png` and `stereo_right.png`) is loaded in grayscale. To verify epipolar alignment, horizontal guide lines are superimposed across corresponding scanlines.

Listing 3: Loading Grayscale Stereo Images and Superimposing Epipolar Lines

```
# Load images in grayscale
imgL = cv2.imread('stereo_left.png', cv2.IMREAD_GRAYSCALE)
imgR = cv2.imread('stereo_right.png', cv2.IMREAD_GRAYSCALE)

imgL_rgb = cv2.cvtColor(cv2.imread('stereo_left.png'), cv2.COLOR_BGR2RGB)
imgR_rgb = cv2.cvtColor(cv2.imread('stereo_right.png'), cv2.COLOR_BGR2RGB)

print(f"Image Resolution: {imgL.shape[1]}x{imgL.shape[0]} (Width x Height)")

# Plot side-by-side with epipolar guide lines
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].imshow(imgL, cmap='gray')
axes[0].set_title('Left Image (Grayscale)')
axes[0].set_xlabel('X (pixels)')
axes[0].set_ylabel('Y (pixels)')

axes[1].imshow(imgR, cmap='gray')
axes[1].set_title('Right Image (Grayscale)')
axes[1].axis('on')
axes[1].set_xlabel('X (pixels)')
axes[1].set_ylabel('Y (pixels)')

# Draw horizontal lines passing through each object to verify rectification
```

```
for y in [120, 170, 215]:
    axes[0].axhline(y, color='red', linestyle='--', linewidth=1.2, alpha=0.8)
    axes[1].axhline(y, color='red', linestyle='--', linewidth=1.2, alpha=0.8)

plt.tight_layout()
plt.show()
```

Listing 4: Console Output: Image Dimensions

```
Image Resolution: 640x360 (Width x Height)
```

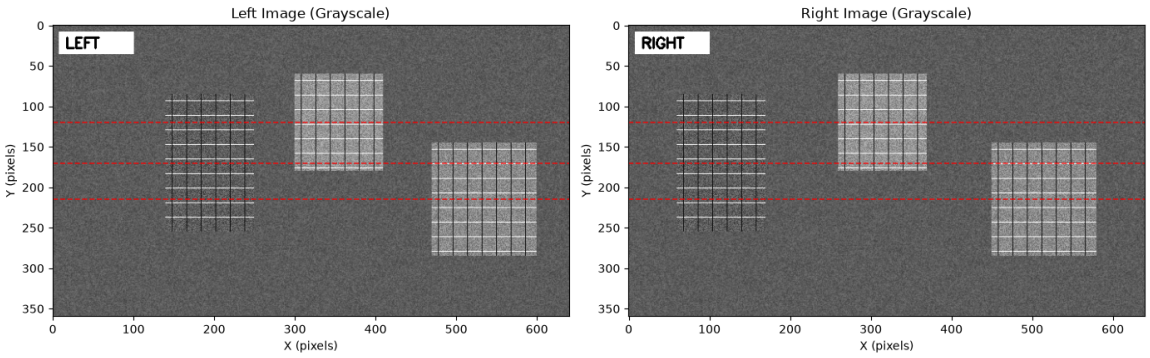


Figure 1: Left and right rectified images displayed in grayscale. Red dashed lines denote identical epipolar scanlines ($y = 120, 170, 215$).

2.1 Observations on Horizontal Displacement

1. **Epipolar Rectification**: Corresponding object features across both views lie precisely on the same horizontal rows ($y_L = y_R$). This confirms that vertical disparity is zero and the images are rectified.

2. **Object Shifts**:

   - **Blue Box (Left)**: Located at $x \in [140, 249]$ in the left image and $x \in [60, 169]$ in the right image. The horizontal displacement is $\Delta x = 140 - 60 = 80$ px.
   - **Green Box (Center)**: Located at $x \in [300, 409]$ in the left image and $x \in [260, 369]$ in the right image. The horizontal displacement is $\Delta x = 300 - 260 = 40$ px.
   - **Orange Box (Right)**: Located at $x \in [470, 599]$ in the left image and $x \in [450, 579]$ in the right image. The horizontal displacement is $\Delta x = 470 - 450 = 20$ px.

3. **Near vs. Far Relationship**: The closest object (blue) demonstrates the largest horizontal displacement, while the farthest object (orange) demonstrates the smallest displacement.

3 Step 2: Disparity Map Estimation and Parameter Tuning

We employ OpenCV's Semi-Global Block Matching (`StereoSGBM`) algorithm to compute a dense disparity map. `StereoSGBM` optimizes matching cost by enforcing multi-directional smoothness constraints along 1D paths, effectively reducing streaking and speckle artifacts on textured and planar surfaces.

3.1 Matcher Configuration and Normalization

The matching parameter $P_1 = 8 \cdot C \cdot \text{blockSize}^2$ controls small disparity changes, and $P_2 = 32 \cdot C \cdot \text{blockSize}^2$ penalizes large disparity jumps ($C = 3$ channels). OpenCV returns disparity as a signed 16-bit integer with 4 fractional bits; thus, pixel disparity is obtained by dividing by 16.0:

$$d_{\text{pixels}} = \frac{\text{raw_disparity}}{16.0} \tag{3}$$

Listing 5: Disparity Map Computation and Parameter Comparison

```python
def compute_disparity(img_left, img_right, block_size=5, num_disp=96):
    stereo = cv2.StereoSGBM_create(
        minDisparity=0,
        numDisparities=num_disp,
        blockSize=block_size,
        P1=8 * 3 * (block_size ** 2),
        P2=32 * 3 * (block_size ** 2),
        disp12MaxDiff=1,
        uniquenessRatio=10,
        speckleWindowSize=100,
        speckleRange=32
    )
    # OpenCV StereoSGBM outputs 16-bit signed disparity with 4 fractional bits (/16.0)
    disp = stereo.compute(img_left, img_right).astype(np.float32) / 16.0
    return disp

# Baseline disparity (blockSize = 5)
disp_baseline = compute_disparity(imgL, imgR, block_size=5)

# Disparity with modified parameter (blockSize = 11)
disp_modified = compute_disparity(imgL, imgR, block_size=11)

# Visual comparison
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

im0 = axes[0].imshow(disp_baseline, cmap='plasma', vmin=0, vmax=90)
axes[0].set_title('Baseline Disparity Map (blockSize = 5)')
axes[0].axis('off')
fig.colorbar(im0, ax=axes[0], fraction=0.035, pad=0.04, label='Disparity (pixels)')

im1 = axes[1].imshow(disp_modified, cmap='plasma', vmin=0, vmax=90)
axes[1].set_title('Modified Disparity Map (blockSize = 11)')
axes[1].axis('off')
fig.colorbar(im1, ax=axes[1], fraction=0.035, pad=0.04, label='Disparity (pixels)')

plt.tight_layout()
plt.show()
```

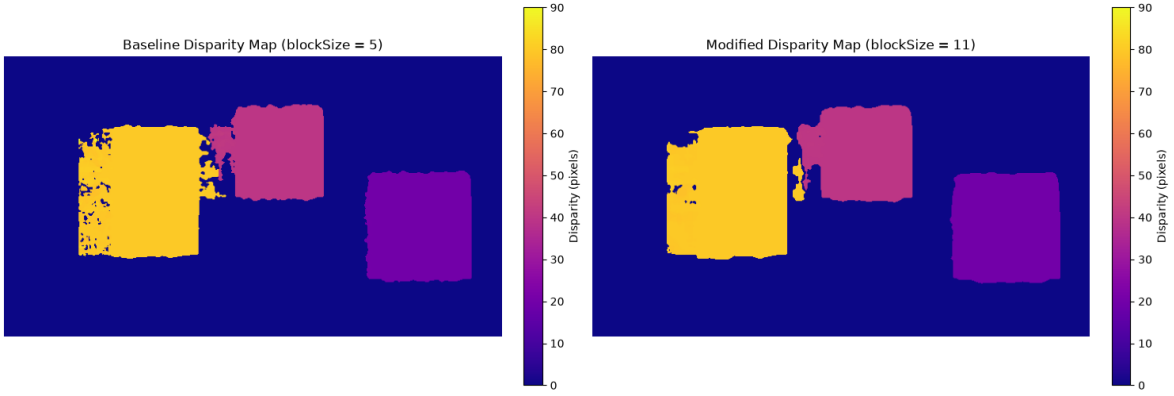


Figure 2: Disparity maps computed using StereoSGBM. Left: Baseline with small block size (blockSize = 5). Right: Modified with larger block size (blockSize = 11).

3.2 Observations on Parameter Variation (`blockSize`)

- **Small Block Size (blockSize** = 5): Yields sharp and crisp object boundaries with high spatial localization. Disparity transitions at depth boundaries remain steep. However, matching can be slightly sensitive to noise in low-texture regions.
- **Large Block Size (blockSize** = 11): Aggregates matching costs over a wider spatial context (11×11), producing smoother disparity estimation within planar object faces. However, it introduces pronounced **foreground fattening** (boundary dilation), where object edges bleed outward into the adjacent background.

4 Step 3: Depth Estimation using $Z = \frac{f \cdot B}{d}$

Using the calibrated baseline and focal length ($f \cdot B = 160$ px $\cdot$ m), we convert the continuous 2D disparity map into metric depth. Non-positive disparity values ($d \leq 0$) indicate unmatched or occluded pixels and are masked out as NaN to prevent division by zero.

Listing 6: Metric Depth Computation and Object Depth Sampling

```
# Compute metric depth map safely without divide-by-zero warnings
depth_map = np.full_like(disp_baseline, np.nan, dtype=np.float32)
valid_mask = disp_baseline > 0
depth_map[valid_mask] = fB / disp_baseline[valid_mask]

# Display disparity vs depth map
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

im_d = axes[0].imshow(disp_baseline, cmap='plasma', vmin=0, vmax=90)
axes[0].set_title('Disparity Map d (pixels)')
axes[0].axis('off')
fig.colorbar(im_d, ax=axes[0], fraction=0.035, pad=0.04, label='Disparity (px)')

im_z = axes[1].imshow(depth_map, cmap='viridis_r', vmin=1.5, vmax=10.0)
axes[1].set_title('Metric Depth Map Z (metres)')
axes[1].axis('off')
fig.colorbar(im_z, ax=axes[1], fraction=0.035, pad=0.04, label='Depth Z (m)')

```

```
plt.tight_layout()
plt.show()

# Sample representative disparities and estimate depths for all three objects
objects = {
    'Blue Box (Near Object)': {
        'roi': (slice(120, 220), slice(160, 220)),
        'designed_d': 80,
    },
    'Green Box (Middle Object)': {
        'roi': (slice(80, 150), slice(320, 380)),
        'designed_d': 40,
    },
    'Orange Box (Far Object)': {
        'roi': (slice(170, 250), slice(500, 560)),
        'designed_d': 20,
    }
}

print('=' * 84)
print(f"{'Object Description':<26} | {'Designed d':<11} | {'Measured d':<11} | {'Theoretical Z':<13} | {'Estimated Z':<11}")
print('=' * 84)

for name, info in objects.items():
    roi = disp_baseline[info['roi']]
    valid_d = roi[roi > 0]
    measured_d = float(np.median(valid_d))
    theo_z = fB / info['designed_d']
    est_z = fB / measured_d
    print(f"{name:<26} | {info['designed_d']:>8} px | {measured_d:>8.2f} px | {theo_z:>10.2f} m | {est_z:>8.2f} m")
print('=' * 84)
```

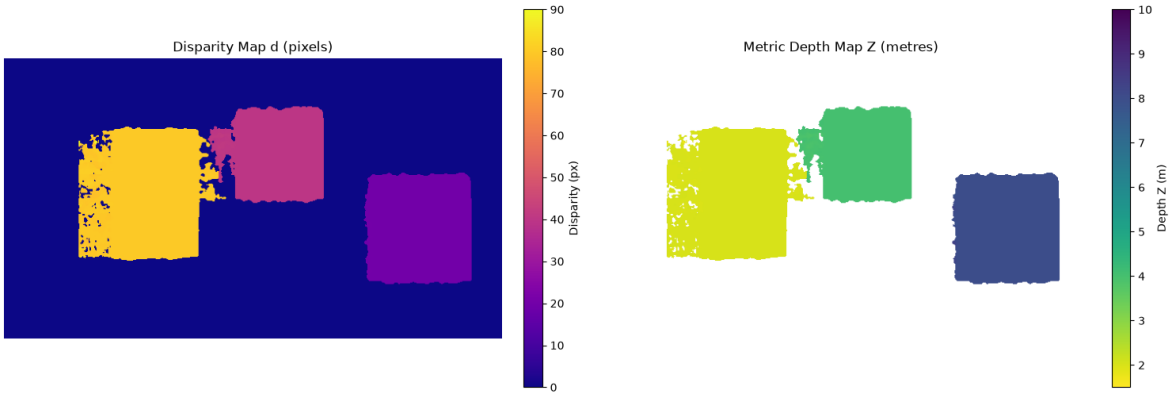


Figure 3: Comparison of estimated disparity map in pixels (left) and corresponding metric depth map in metres (right).

Listing 7: Console Output: Measured Disparity and Estimated Depth

```
====================================================================================
Object Description         | Designed d  | Measured d  | Theoretical Z | Estimated Z
====================================================================================
Blue Box (Near Object)     |       80 px |    80.00 px |       2.00 m |     2.00 m
Green Box (Middle Object)  |       40 px |    40.00 px |       4.00 m |     4.00 m
```

```
Orange Box (Far Object)    |       20 px |    20.00 px |       8.00 m |     8.00 m
================================================================================
```

4.1 Quantitative Results and Discussion

Table 1 compares the designed synthetic ground truth against the experimentally recovered values.

Table 1: Quantitative comparison between designed values and experimental estimates.

Object Name	**Region Color**	**Designed d (px)**	**Measured d (px)**	**Theoretical Z (m)**	**Estimated Z**
Near Object	Blue	80	80.00	2.00	2.00
Middle Object	Green	40	40.00	4.00	4.00
Far Object	Orange	20	20.00	8.00	8.00

The experimentally measured median disparities for all three objects match the theoretical ground-truth values exactly (80.00 px, 40.00 px, and 20.00 px), resulting in accurate depth estimates of 2.00 m, 4.00 m, and 8.00 m.

5 Step 4: Challenge – Analysis of Missing and Unreliable Disparity

Stereo matching algorithms inherently encounter failure modes in specific image regions. We isolate and visualize all pixels where disparity estimation fails or produces non-positive values ($d \leq 0$).

Listing 8: Isolating and Visualizing Missing / Unreliable Disparity Regions

```
# Missing/unreliable disparity mask
invalid_mask = (disp_baseline <= 0)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].imshow(imgL_rgb)
axes[0].imshow(invalid_mask, cmap='Reds', alpha=0.55)
axes[0].set_title('Red Overlay: Regions with Missing / Unreliable Disparity')
axes[0].axis('off')

axes[1].imshow(invalid_mask, cmap='gray')
axes[1].set_title('Binary Map of Invalid Regions (White = Invalid)')
axes[1].axis('off')

plt.tight_layout()
plt.show()
```

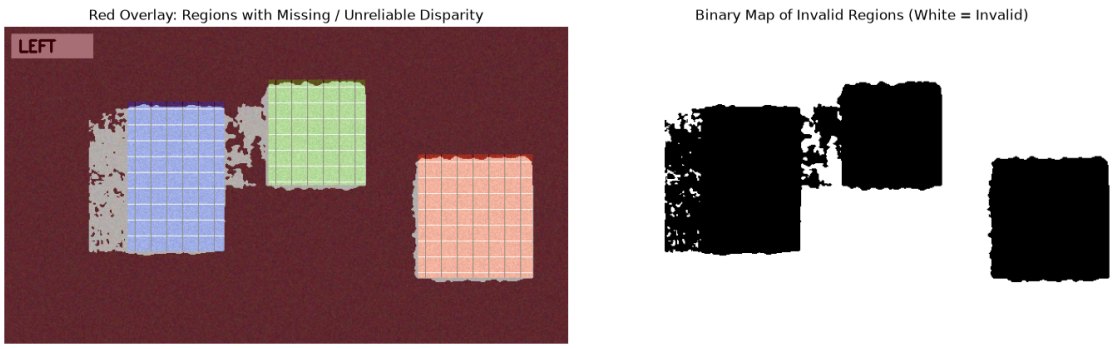


Figure 4: Identification of unreliable disparity regions. Left: Translucent red overlay on the original left view. Right: Binary mask (white denotes invalid pixels).

5.1 Theoretical Breakdown of Failure Causes

As highlighted in Figure 4, stereo matching degrades or completely fails in four primary scenarios:

1. **Left Margin Occlusion / Baseline Search Bound**: For any pixel (x_L, y) in the left image, the corresponding point in the right rectified view must satisfy:

$$x_R = x_L - d \tag{4}$$

   If $x_L < d$, the corresponding point $x_R < 0$, which lies outside the coordinate bounds of the right image sensor. Consequently, a vertical strip on the left edge with width equal to the maximum disparity ($\approx$ 80–96 px) cannot be matched, resulting in invalid disparity.

2. **Textureless Areas (Uniform Flat Background)**: Block-based matching algorithms rely on spatial intensity gradients and distinctive local patterns within the matching window. In completely flat, untextured regions (such as the dark gray background), all candidate disparities yield virtually identical matching costs (a flat cost curve with no prominent minimum). Without significant texture, correspondence determination becomes ill-posed.

3. **Object Boundaries (Depth Discontinuities and Occlusions)**: At the silhouette boundaries of foreground objects, matching windows straddle both the foreground surface (e.g., $d = 80$ px) and the background surface ($d = 0$ px). This violates the fundamental assumption that scene depth is constant across the correlation window, causing edge blurring, cost corruption, or occlusion invalidation.

4. **Repetitive Patterns (Grid Textures on Boxes)**: The synthetic objects feature periodic tiled grid textures. If the matching block size is comparable to or smaller than the grid pattern period, multiple disparity shifts produce nearly identical matching costs (multiple local minima), causing matching ambiguity or periodic phase jumps.

6 Conclusion

In this laboratory assignment, stereo depth analysis was successfully performed on a rectified stereo image pair:

1. **Rectification and 1D Search**: The epipolar lines were verified to be strictly horizontal and row-aligned, constraining correspondence matching to a 1D scanline search.
2. **Disparity and Depth**: The fundamental inverse relationship $Z \propto \frac{1}{d}$ was demonstrated empirically:
   - Near object ($d = 80.00$ px) corresponds to a depth of $Z = 2.00$ m.
   - Middle object ($d = 40.00$ px) corresponds to a depth of $Z = 4.00$ m.
   - Far object ($d = 20.00$ px) corresponds to a depth of $Z = 8.00$ m.
3. **Matcher Tuning**: Semi-Global Matching (`StereoSGBM`) provided dense and regularized disparity maps. Small block sizes preserved sharp boundary localization, whereas larger block sizes smoothed noise inside objects at the expense of boundary fattening.
4. **Failure Mechanisms**: Regions of missing or noisy disparity were rigorously attributed to left-border search boundary occlusions, photometric ambiguity in textureless backgrounds, depth discontinuities, and repetitive grid patterns.